CÔNG TY CỔ PHẦN ICORP
Địa chỉ: Số 32/21 Phố Trương Công Giai, Phường Dịch Vọng, Quận Cầu Giấy, TP Hà Nội
Tel: 1900.0099 Fax: 024.6655.8263 Email: ica@icorp.vn Website: i-ca.vn

HSM SIGNING

API Specification

Version 2.0_22.04.22

CONTENTS

1.INTRODUCTION

This document describes in details the API of Remote Signing service. In this version, API is now including 10 functions:

- prepareCertificateForSignCloud
- prepareFileForSignCloud
- prepareHashSigningForSignCloud
- authorizeCounterSigningForSignCloud
- authorizeSingletonSigningForSignCloud
- authorizeHashSigningForSignCloud
- getCertificateDetailForSignCloud
- changePasscodeForSignCloud
- forgetPasscodeForSignCloud
- regenerateAuthorizationCodeForSignCloud
- uploadFileForSignCloud

1.1. Target

Banks, Finance/Insurance companies who wants to apply digital signature for loan approval.

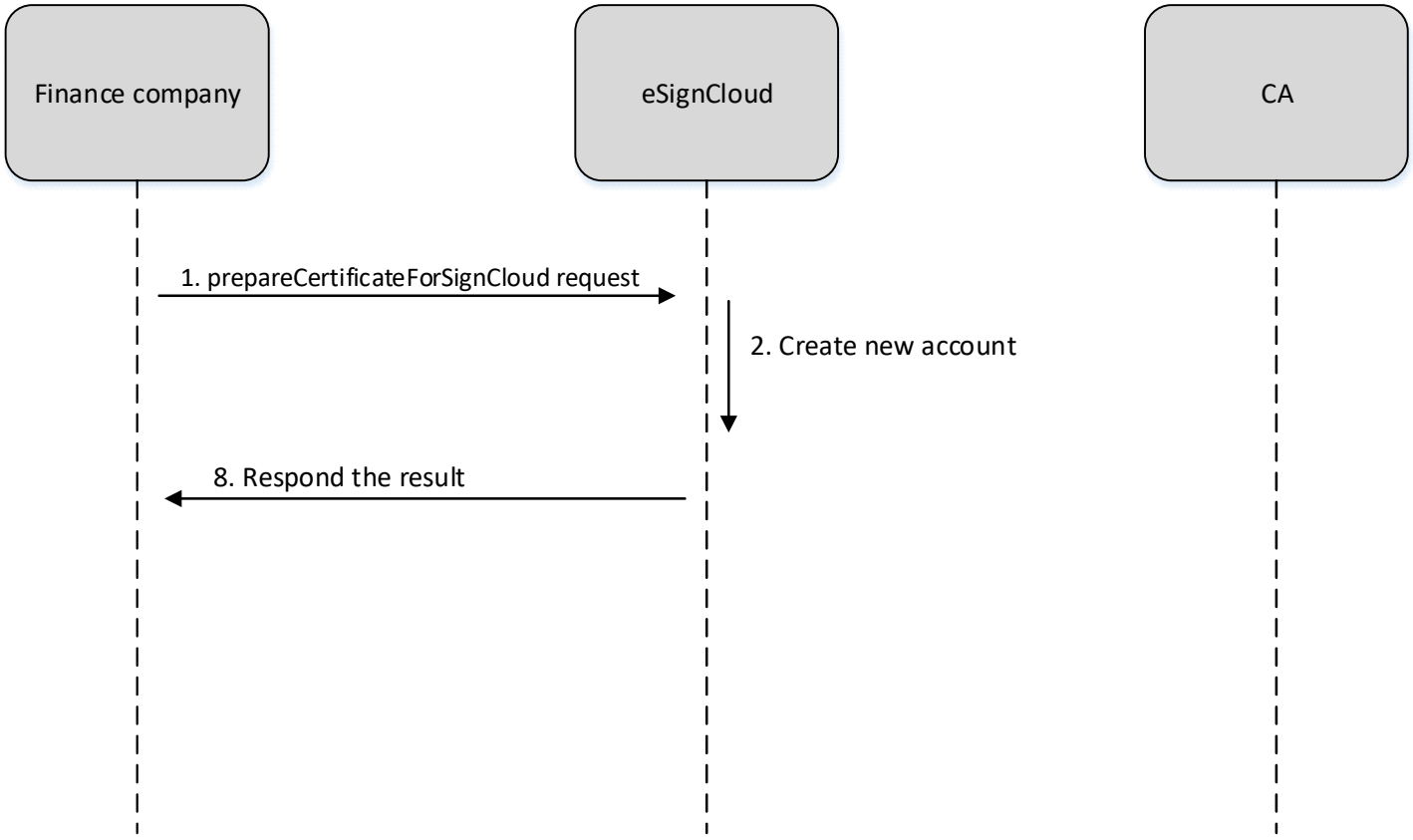
1.2. Abbreviation

Word	Description
CA	Certification Authority
PKI	Public Key Infrastructure
M	Mandatory
O	Optional
M/O	Mandatory/Optional depends on each specific case
AP	Application Provider
RP	Relying Party

2. API SPECIFICATION

2.1. prepareCertificateForSignCloud

This method will create a new account on Remote Signing service and request to CA for preparing digital certificate.



2.1.1. Prototype

```
SignCloudResp prepareCertificateForSignCloud(SignCloudReq signCloudReq);
```

2.1.2. Parameters

SignCloudReq Attributes				
No	Name	Type	Require	Description
1	relyingPartyBillCode	String	O	Relying Party's billcode
2	relyingParty	String	M	In case of many services are running on Remote Signing. RelyingParty is used to determine which client is calling to.
3	agreementUUID	String	M	Identify the client/customer/agreement number
4	mobileNo	String	O	Mobile Number of client/customer. Known as contact address and used for SMS Authorize Code authentication.

SignCloudReq Attributes				
No	Name	Type	Require	Description
				In case of the Relying Party keep the customer information confidentially, it should be absence
5	email	String	O	Email of client/customer and used for Email Authorize Code authentication In case of the Relying Party keep the customer information confidentially, it should be absence
6	certificateProfile	String	O	The profile of certificate will be used to enroll. Once this parameter is absence, the default certificate profile is used based on RelyingParty properties
7	agreementDetails	AgreementDetails	M	Customer information. This one will be used as certificate information.
8	credentialData	CredentialData	M	Credential information used for client-server handshake Cached feature is turn **ON/OFF**. Once the cached machine is **ON**, the Remote Signing just checked the credentialData in the handshake phase. Once credentialData is valid, our session will be valid in duration of 1 hour (this parameter could be reconfigured in Remote Signing service). The Relying Party need to handshake again before the session is expired

In case of AgreementDetails, (*) is mandatory and (/*) is at least ONE in mandatory

AgreementDetails Attributes				
No	Name	Type	Require	Description
1	**personName (*)**	String	M/O	Name of customer
2	organization	String	M/O	Company name
3	organizationUnit	String	M/O	Department nam
4	title	String	M/O	Title of employee
5	email	String	M/O	Personal email
6	telephoneNumber	String	M/O	Personal phone number

AgreementDetails Attributes				
No	Name	Type	Require	Description
7	**location (*)**	String	M	
8	**stateOrProvince (*)**	String	M	City or Province
9	country	String	M	Country (Just two letter) E.g: VN
10	**personalID (/*)**	String	M/O	Personal ID
	citizenID (/*)	String	M/O	Citizen ID
	passportID (/*)	String	M/O	Personal Passport ID
11	taxID (/*)	String	M/O	Tax ID
	budgetID (/*)	String	M/O	Budget ID
12	applicationForm	Byte[]	M/O	Remote Signing Service application form
13	requestForm	Byte[]	M/O	Remote Signing Service request form for recovery, revocation …
14	authorizeLetter	Byte[]	M/O	Authorize Letter by Government/Corporate
15	photoIDCard	Byte[]	M/O	Scanned photo of ID Card that applied for personal certificate (In case only 1 file available)
16	photoFrontSideIDCard	Byte[]	M/O	Scanned front side of photo of ID Card that applied for personal certificate
17	photoBackSideIDCard	Byte[]	M/O	Scanned back side side of photo of ID Card that applied for personal certificate
18	photoActivityDeclaration	Byte[]	M/O	Scanned photo of 'Initial Activity/Alteration Declation' that applied for corporate certificate

SignCloudResp Attributes				
No	Name	Type	Require	Description
1	responseCode	int	M	Code describes the result. Normally. Remote Signing will response **SUCCESSFULLY**
2	responseMessage	String	M	Message describes the result in detail.
3	billCode	String	M	Receipt for each transaction
4	certificateDN	String	M/O	Certificate Distinguished Name (DN) ***E.g: CN=Nguyen Van A, O=B Company,C=VN***
5	certificateSerialNumber	String	M/O	Certificate serial number E.g: 5405ABCDEF
6	certificateThumbprint	String	M/O	Certificate thumprint (certificate hash)

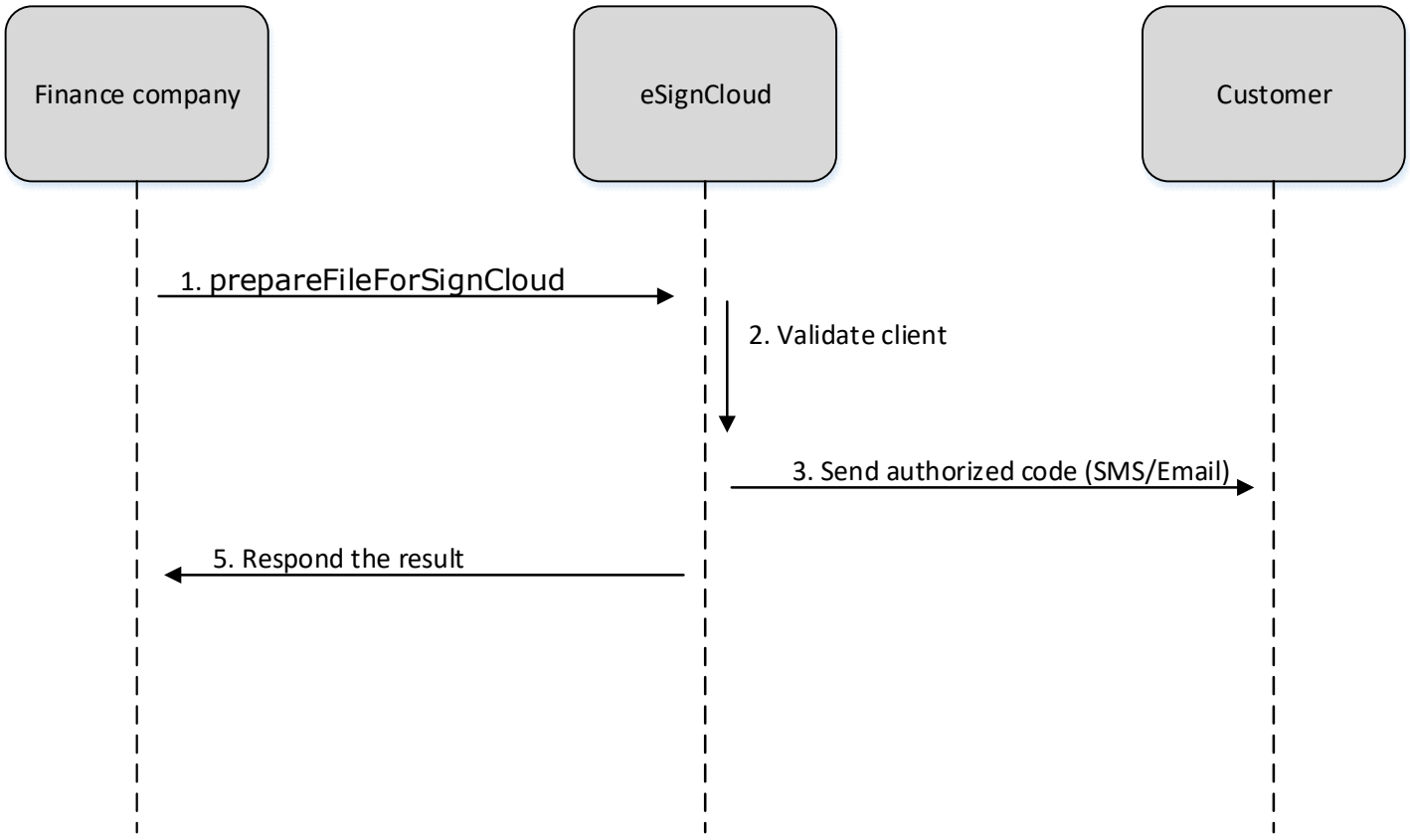
SignCloudResp Attributes				
No	Name	Type	Require	Description
7	validFrom	Date	M/O	The date of certificate begin to be valid
8	validTo	Date	M/O	The date of certificate begin to be expired
9	issuerDN	String	M/O	Issuer Certifcate Distinguished Name (DN) ***E.g: CN=ICA Certification Authority, O=I-CA,C=VN***
10	certificate	String	M/O	The data of certificate in PEM format
11	certificateStateID	int	M/O	Certificate state ID • INITIALIZED = 2 • GENERATED = 3 • OPERATED = 4 • REVOKED = 5 • EXPIRED = 6 • DECLINED = 7 • RENEWED = 8 • RENEWED_EXPIRED = 9 • REVISED = 10 • RENEWED_KEEP_SN = 11 • BLOCKED = 12 • REVISED_KEEP_SN = 13
12	signingCounter	int	M/O	How many times the certificate is used for signing. The default value is 1. For document signing this value should be greater than 1

CredentialData Attributes				
No	Name	Type	Require	Description
1	username	String	M	Username of relyingParty provided by Remote Signing
2	password	String	M	Password of relyingParty provided by Remote Signing
3	signature	String	M	Signature of relyingParty provided by Remote Signing
4	pkcs1Signature	String	M	Value is signed from client private key. Key is generated and provided by Remote Signing Value=username + password + signature + timestamp
5	timestamp	String	O	Current timestamp, format in yyyyddmmHHMMss or epoch time

CredentialData Attributes				
No	Name	Type	Require	Description
				If timestamp is absence, the pkcs1Signature is fixed for all transactions

2.2. prepareFileForSignCloud

This method is known as "Sign Request", it means that client request to Remote Signing. Remote Signing will send back an Authorize Code via SMS or Email to authorize the client. This method also requires the binary of document which needs to be signed.



2.2.1. Prototype

```
SignCloudResp prepareFileForSignCloud(SignCloudReq signCloudReq);
```

2.2.2. Parameters

SignCloudReq Attributes				
No	Name	Type	Require	Description
1	relyingPartyBillCode	String	O	Relying Party's billcode
2	relyingParty	String	M	In case of many services are running on Remote Signing.

SignCloudReq Attributes				
No	Name	Type	Require	Description
				RelyingParty is used to determine which client is calling to.
3	agreementUUID	String	M	Identify the client/customer/agreement ID
4	authorizeMethod	int	O	Which method will be used to authorize to client. 1: OTP SMS (default) 2: OTP Email 3: OTP Mobile 4: Passcode 5: UAF *This attribute will be deprecated in the future and replaced by authMode*
5	signingFileData	Byte[]	M/O	File binary that will be signed
6	signingFileName	String	M	File name
7	mimeType	String	M/O	In case of PDF signer, mimeType is '**application/pdf**'
8	xmlDocument	String	M/O	XML Document is customer data that need to be signed
9	multipleSigningFileData	List<MultipleSigningFileData>	M/O	In case of multiple files need to be signed by one time authentication. This object contains a list of document will be signed.
10	notificationTemplate	String	M	Message contains Authorize Code will be sent to customer's Phone/Email. Authorize Code is generated on Remote Signing and embedded into template. E.g: Your authorize code: **{AuthorizeCode}**. Authorize Code is valid within 5 minutes.
11	notificationSubject	String	M/O	This parameter is used as Email subject for signing authorization.

SignCloudReq Attributes				
No	Name	Type	Require	Description
				If OTP Email is used, this parameter is mandatory It should be used in case of the Remote Signing will use owned SMTP to send OTP Email to customer.
12	signCloudMetaData	signCloudMetaData	O	SignCloudMetaData contains additional signer properties which are used for PDF signature display. - Coordinate - Background image - Signer location - Signer reason - Text color In case of XML sigining, it will indicate which type should be applied XMLDSig, XML-XaDES, signature zone Once this parameter is not used, Remote Signing should use the default value that already configured in the system In case of counter signing process, SignCloudMetaData included 2 MetaDatas for customer and Relying Party signer properties
13	authorizeCode	String	M/O	Authorize Code provided by customer In this case of passcode (4) is used, authorize code is passcode
14	messagingMode	int	M/O	Which messaging mode will be used

SignCloudReq Attributes				
No	Name	Type	Require	Description
				1: ASYNCHRONOUS_CLIENTSERVER (default) **2**: ASYNCHRONOUS_SERVERSERVER **3**: SYNCHRONOUS If default value 0 is used, the API downloadSignedFileForSignCloud will be used for getting the signed file
15	credentialData	CredentialData	M	Credential information used for client-server handshake

SignCloudResp Attributes				
No	Name	Type	Require	Description
1	responseCode	int	M	Code describes the result.
2	responseMessage	String	M	Message describes the result in detail.
3	billCode	String	M	Receipt for each transaction
4	notificationMessage	String	M/O	In case of Authorize Code isn't sent by Remote Signing, notification message which has already included Authorize Code will be sent back to Finance/Insurance company to send to their end-user.
5	notificationSubject	String	M/O	This parameter is used as Email subject for signing authorization. It should be used in case of RelyingParty used their owned SMTP server for notify customer
6	authorizeCredential	String	M/O	In case of Authorize Code isn't sent by Remote Signing,

SignCloudResp Attributes				
No	Name	Type	Require	Description
				Email/Mobile number of client/customer also is responded back to Finance/Insurance company and they use that information to send Authorize Code to their end-user.
7	signedFileData	Byte[]	M/O	File binary that be signed
8	mimeType	String	M/O	In case of PDF file that be downloaded, mimeType is **'application/pdf'**
9	multipleSignedFileData	List<MultipleSignedFileData>	M/O	In case of signing multiple files, the multiple signed files will be responded in this object.
10	xmlSignature	String	M/O	XML Signature (XMLDSig or XML-XadDES)
11	certificateDN	String	M/O	Certificate Distinguished Name (DN) ***E.g: CN=Nguyen Van A, O=B Company,C=VN***
12	certificateSerialNumber	String	M/O	Certificate serial number E.g: 5405ABCDEF
13	certificateThumbprint	String	M/O	Certificate thumprint (certificate hash)
14	validFrom	Date	M/O	The date of certificate begin to be valid
15	validTo	Date	M/O	The date of certificate begin to be expired
16	issuerDN	String	M/O	Issuer Certifcate Distinguished Name (DN) ***E.g: CN=Mobile-ID Trusted Network, O=MOBILE-ID,C=VN***

MultipleSigningFileData Attributes				
No	Name	Type	Require	Description
1	signingFileData	Byte[]	M/O	The binary of document
2	signingFileName	String	M/O	The document name
3	mimeType	String	M/O	The document mime type. The possible values: • application/pdf • application/xml • application/xhtml+xml • application/msword • application/ vnd.openxmlformats-officedocument.wordprocessingml.document • application/vnd.ms-powerpoint • application/vnd.openxmlformats-officedocument.presentationml.presentation • application/vnd.ms-excel • application/vnd.openxmlformats-officedocument.spreadsheetml.sheet • application/vnd.visio
4	xmlDocument	String	M/O	The document in xml format.
5	hash	String	M/O	Hash to be signed
6	signCloudMetaData	SignCloudMetaData	M/O	SignCloudMetaData contains additional signer properties which are used for PDF signature display. - Coordinate - Background image - Signer location - Signer reason - Text color In case of XML sigining, it will indicate which type should be applied XMLDSig, XML-XaDES, signature zone Once this parameter is not used, Remote Signing should use the default value that already configured in the system

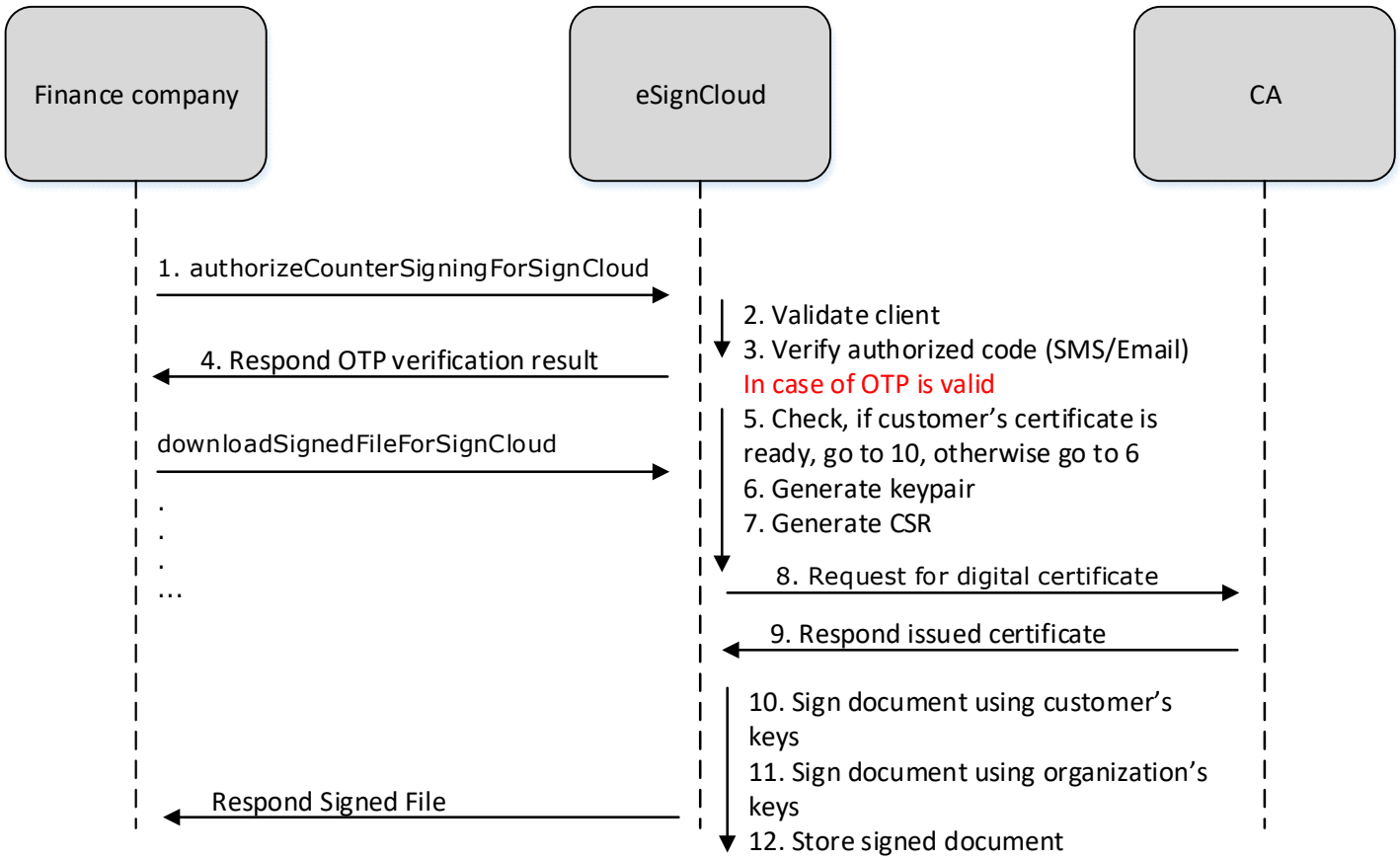
MultipleSigningFileData Attributes				
No	Name	Type	Require	Description
				In case of counter signing process, SignCloudMetaData included 2 MetaDatas for customer and Relying Party signer properties

MultipleSignedFileData Attributes				
No	Name	Type	Require	Description
1	signedFileData	Byte[]	M/O	The signed document binary
2	mimeType	String	M/O	The signed document mime type
3	signedFileName	String	M/O	The signed document name
4	signatureValue	String	M/O	The PKCS#1 Signature as Base64 String will be sent to RelyingParty

2.3. authorizeCounterSigningForSignCloud

This method is known as "Sign Response", that means client responds the Authorize Code to Remote Signing. Remote Signing will verify the received Authorize Code and response signed document (contract) whether it's correct. In case of the agreement don't own the certificate, the Remote Signing will enroll the corresponding ceritificate for business optimization and later will sign the document

Once the customer's private key is used for document signing, the organization private key that corresponded to the RelyingParty owned will be used for document co-signing process



2.3.1. Prototype

```
SignCloudResp authorizeCounterSigningForSignCloud(SignCloudReq signCloudReq);
```

2.3.2. Parameters

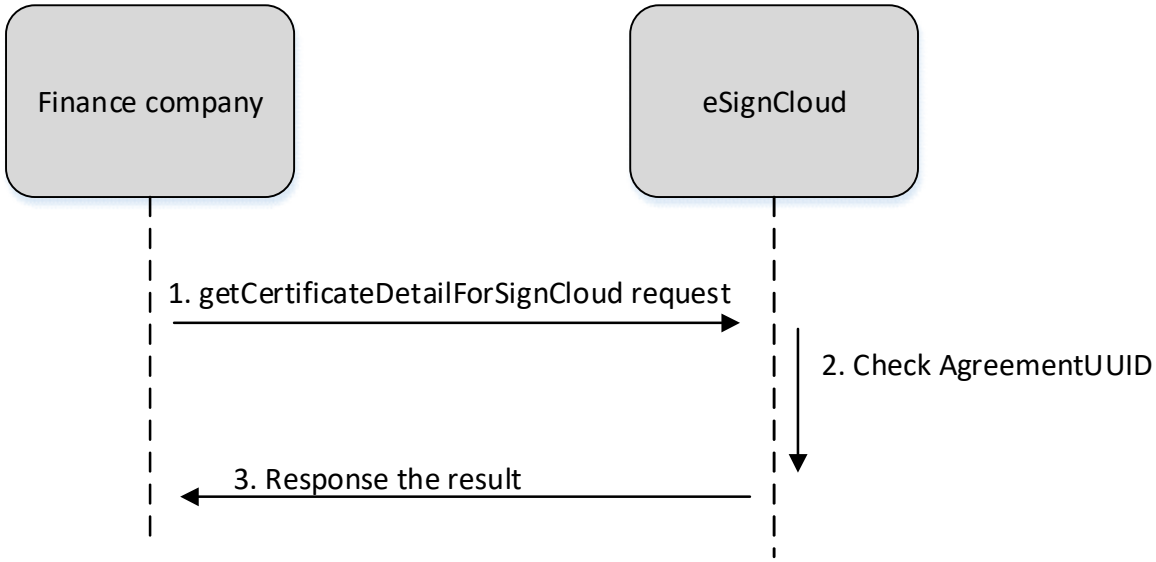
SignCloudReq Attributes				
No	Name	Type	Require	Description
1	relyingPartyBillCode	String	O	Relying Party's billcode
2	relyingParty	String	M	In case of many services are running on Remote Signing. RelyingParty is used to determine which client is calling to.
3	agreementUUID	String	M	Identify the client/customer/agreement ID
4	billCode	String	M	billCode obtained from response of **prepareFileForSignCloud**
5	authorizeCode	String	M	Authorize Code provided by customer
6	messagingMode	int	O	Which messaging mode will be used **1**: ASYNCHRONOUS_CLIENTSERVER (default) **2**: ASYNCHRONOUS_SERVERSERVER **3**: SYNCHRONOUS

SignCloudReq Attributes				
No	Name	Type	Require	Description
				If default value 0 is used, the API downloadSignedFileForSignCloud will be used for getting the signed file
7	credentialData	CredentialData	M	Credential information used for client-server handshake

SignCloudResp Attributes				
No	Name	Type	Require	Description
1	responseCode	int	M	Code describes the result.
2	responseMessage	String	M	Message describes the result in detail. Normally, this responses **SUCCESSFULLY**
3	billCode	String	M	Receipt for each transaction
4	remainingCounter	int	M	Counter decreases once invalid Authorization Code provided. AgreementUUID will be blocked if counter goes to zero and it's automatically unblocked within 5 minutes. The default value for automatic unblock is 5 minutes, it could be reconfigured in the system
5	signedFileData	Byte[]	M/O	File binary that be signed
6	mimeType	String	M/O	In case of PDF file that be downloaded, mimeType is '**application/pdf'**
7	signedFileUUID	String	M/O	Signed file UUID that identified by our Remote Signing service
8	multipleSignedFileData	List<MultipleSignedFileData>	M/O	In case of signing multiple files, the multiple signed files will be responded in this object.
9	xmlSignature	String	M/O	XML Signature (XMLDSig or XML-XadDES)

2.4. getCertificateDetailForSignCloud

This method will get certificate detail for corresponding agreement in Remote Signing service



2.4.1. Prototype

```
SignCloudResp getCertificateDetailForSignCloud(SignCloudReq signCloudReq);
```

2.4.2. Parameters

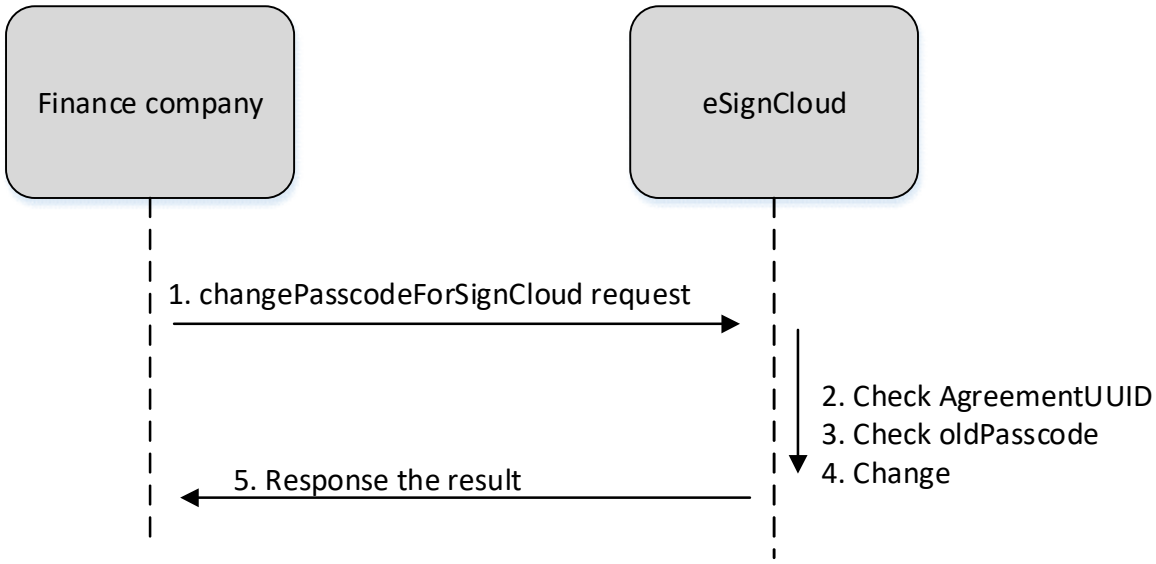
SignCloudReq Attributes				
No	Name	Type	Require	Description
1	relyingPartyBillCode	String	O	Relying Party's billcode
2	relyingParty	String	M	In case of many services are running on Remote Signing. RelyingParty is used to determine which client is calling to.
3	agreementUUID	String	M	Identify the client/customer/agreement number
4	credentialData	CredentialData	M	Credential information used for client-server handshake
5	agreementDetails	AgreementDetails	O	Customer information. It is required for signer double checking before the detail of certificate information responded. One of below information should be provided: + personalName + personalID or citizenID or passportID + organization + taxID or budgetID

SignCloudResp Attributes				
No	Name	Type	Require	Description
1	responseCode	int	M	Code describes the result.
2	responseMessage	String	M	Message describes the result in detail.
3	billCode	String	M	Receipt for each transaction
4	certificateDN	String	M/O	Certificate Distinguished Name (DN) ***E.g: CN=Nguyen Van A, O=B Company,C=VN***
5	certificateSerialNumber	String	M/O	Certificate serial number E.g: 5405ABCDEF
6	certificateThumbprint	String	M/O	Certificate thumprint (certificate hash)
7	validFrom	Date	M/O	The date of certificate begin to be valid
8	validTo	Date	M/O	The date of certificate begin to be expired
9	issuerDN	String	M/O	Issuer Certifcate Distinguished Name (DN) ***E.g: CN=Mobile-ID Trusted Network, O=MOBILE-ID,C=VN***
10	certificate	String	M/O	The data of certificate in PEM format
11	authModeSupported	String[]	M/O	Specifies one of the authorization modes. ▪ "EXPLICIT/PIN": the authorization process is managed by the signature application, authentication method is passcode. ▪ "EXPLICIT/OTP-SMS": the authorization process is managed by the signature application, authentication method is otp sms. ▪ "EXPLICIT/OTP-EMAIL": the authorization process is managed by the signature application, authentication method is otp email. ▪ "IMPLICIT/CYBER-ID": the authorization process is managed by the remote service autonomously. Authentication factors are managed by the RSSP by interacting directly with the user, and not by the signature application. ▪ "IMPLICIT/BIP-CATTP": the authorization process is managed by Cyber-ID – a mobile application – which could interact to PKI USIM for signing.

SignCloudResp Attributes				
No	Name	Type	Require	Description
				▪ "EXPLICIT/OTP-MOBILE": the authorization process is managed by OTP Mobile Application. ▪ "OAUTH2": not implement yet.

2.5. changePasscodeForSignCloud

This method will change the current Passcode for authorize as Passcode method on the Remote Signing service



2.5.1. Prototype

```
SignCloudResp changePasscodeForSignCloud(SignCloudReq signCloudReq);
```

2.5.2. Parameters

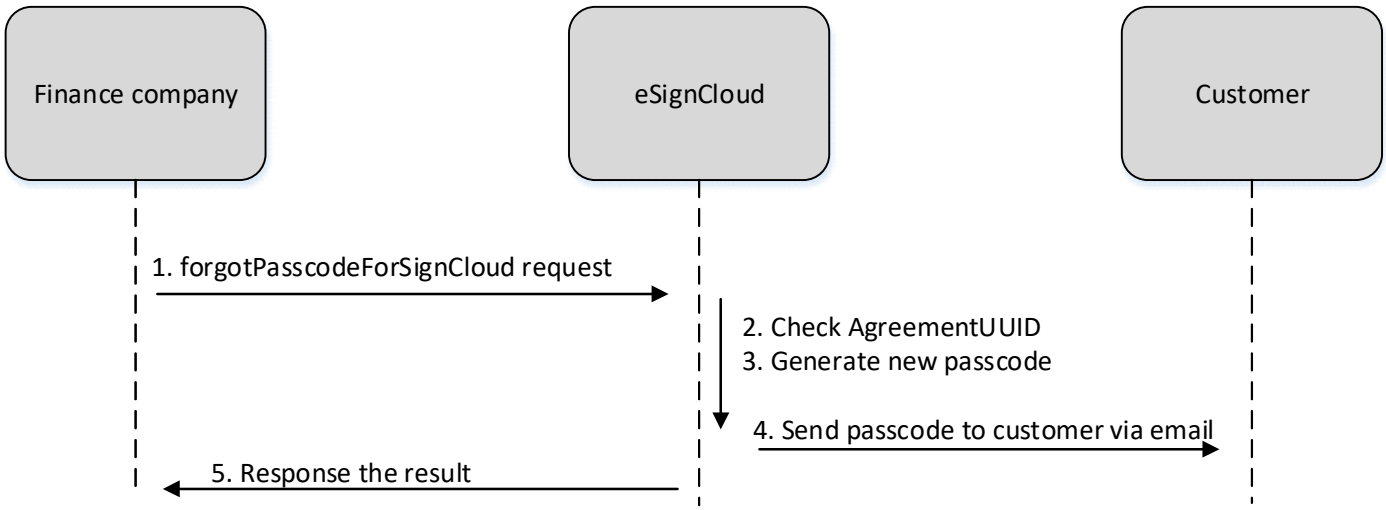
SignCloudReq Attributes				
No	Name	Type	Require	Description
1	relyingPartyBillCode	String	O	Relying Party's billcode
2	relyingParty	String	M	In case of many services are running on Remote Signing. RelyingParty is used to determine which client is calling to.
3	agreementUUID	String	M	Identify the client/customer/agreement ID
4	currentPasscode	String	M	This is the current Passcode that current using for Remote Signing service
5	newPasscode	String	M	This is the new Passcode that will be used later.

SignCloudReq Attributes				
No	Name	Type	Require	Description
				It should be noted the RelyingParty check new Passcode confirmation for ensure that customer owned the new Passcode
6	credentialData	CredentialData	M	Credential information used for client-server handshake

SignCloudResp Attributes				
No	Name	Type	Require	Description
1	responseCode	int	M	Code describes the result.
2	responseMessage	String	M	Message describes the result in detail.
3	billCode	String	M	Receipt for each transaction
4	remainingCounter	int	M	Once changed Passcode successfully, this parameter will be the Max-Counter that already configured in the General Policy of Remote Signing service Once changed Passcode failed, this parameter will be decreased ONE value, if this parameter reached ZERO, our agreement in Remote Signing will be blocked in the duration that already configured. Once this duration ended, our agreement will be unblocked automatically and we continued to use with current Passcode

2.6. forgetPasscodeForSignCloud

This method will reset the current Passcode for authorize as Passcode method on the Remote Signing service , and the new Passcode will be created in the Remote Signing service and this parameter will be sent to customer based on the registration email



2.6.1. Prototype

```
SignCloudResp forgetPasscodeForSignCloud(SignCloudReq signCloudReq);
```

2.6.2. Parameters

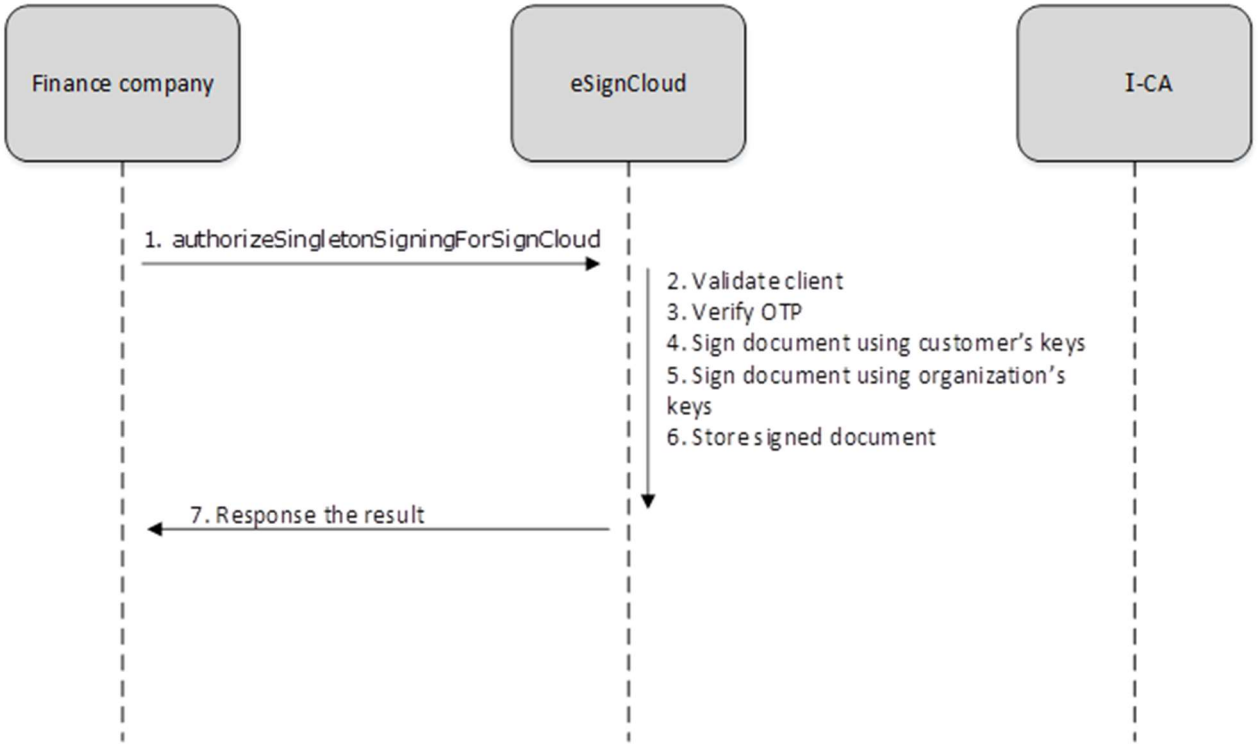
SignCloudReq Attributes				
No	Name	Type	Require	Description
1	relyingPartyBillCode	String	O	Relying Party's billcode
2	relyingParty	String	M	In case of many services are running on Remote Signing. RelyingParty is used to determine which client is calling to.
3	agreementUUID	String	M	Identify the client/customer/agreement ID
3	notificationTemplate	String	M	Message contains Authorize Code will be sent to customer's Email. New Passcode is generated on Remote Signing and embedded into template. E.g: Your new Passcode: **{NewPasscode}**. New Passcode is valid within 30 days.
4	notificationSubject	String	M	This parameter is used as Email subject for forgot Passcode notification. It should be used in case of the Remote Signing will use owned SMTP to send OTP Email to customer.
5	credentialData	CredentialData	M	Credential information used for client-server handshake

SignCloudResp Attributes				
No	Name	Type	Require	Description
1	responseCode	int	M	Code describes the result.

SignCloudResp Attributes				
No	Name	Type	Require	Description
2	responseMessage	String	M	Message describes the result in detail. Normally, our message is **SUCCESSFULLY**
3	billCode	String	M	Receipt for each transaction
4	notificationMessage	String	M/O	Message contains randomly new Passcode will be sent to customer's Email. New Passcode is generated on Remote Signing and embedded into template. E.g: Your new Passcode: **{Passcode}**. New Passcode is valid within 30 days.
5	notificationSubject	String	M/O	This one is used as Email subject for forget Passcode notification. It should be used in case of RelyingParty used their owned SMTP server for notify customer
6	authorizeCredential	String	M/O	In case of Authorize Code isn't sent by Remote Signing, Email of client/customer also is responded back to Finance/Insurance company and they use that information to send Authorize Code to their end-user.

2.7. authorizeSingletonSigningForSignCloud

This method is known as "Sign Response", that means client responds the Authorize Code to Remote Signing. Remote Signing will verify the received Authorize Code and response signed document (contract) whether it's correct. In case of the agreement don't own the certificate, the Remote Signing will enroll the corresponding ceritificate for business optimization and later will sign the document. This method is different from **authorizeCounterSigningForSignCloud** as only one signature will be applied, the counterpart's one won't be.



2.7.1. Prototype

```
SignCloudResp authorizeSingletonSigningForSignCloud(SignCloudReq signCloudReq);
```

2.7.2. Parameters

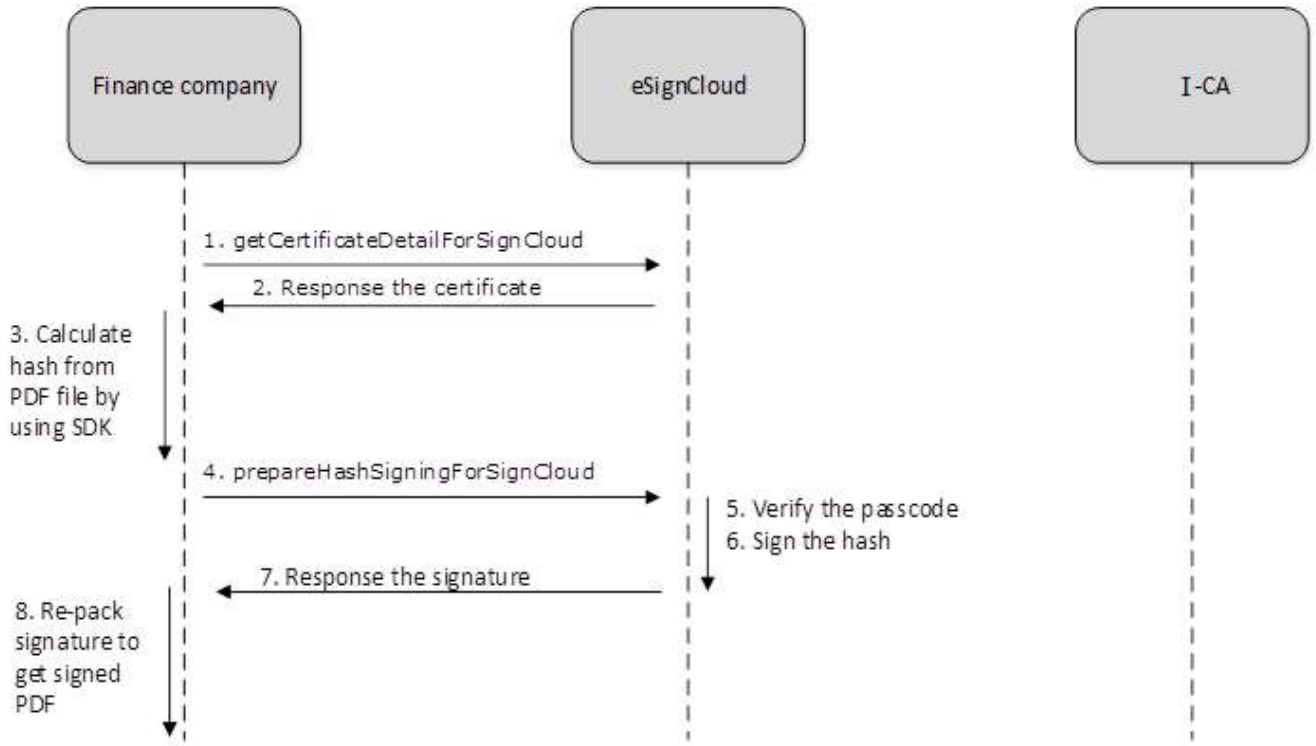
SignCloudReq Attributes				
No	Name	Type	Require	Description
1	relyingPartyBillCode	String	O	Relying Party's billcode
2	relyingParty	String	M	In case of many services are running on Remote Signing. RelyingParty is used to determine which client is calling to.
3	agreementUUID	String	M	Identify the client/customer/agreement ID
4	billCode	String	M	billCode obtained from response of **prepareFileForSignCloud**
5	authorizeCode	String	M	Authorize Code provided by customer
6	messagingMode	int	O	Which messaging mode will be used **1**: ASYNCHRONOUS_CLIENTSERVER (default) **2**: ASYNCHRONOUS_SERVERSERVER **3**: SYNCHRONOUS If default value 0 is used, the API downloadSignedFileForSignCloud will be used for getting the signed file

SignCloudReq Attributes				
No	Name	Type	Require	Description
7	credentialData	CredentialData	M	Credential information used for client-server handshake

SignCloudResp Attributes				
No	Name	Type	Require	Description
1	responseCode	int	M	Code describes the result.
2	responseMessage	String	M	Message describes the result in detail.
3	billCode	String	M	Receipt for each transaction
4	remainingCounter	int	M	Counter decreases once invalid Authorization Code provided. AgreementUUID will be blocked if counter goes to zero and it's automatically unblocked within 5 minutes. The default value for automatic unblock is 5 minutes, it could be reconfigured in the system
5	signedFileData	Byte[]	M/O	Signed document
6	mimeType	String	M/O	In case of PDF signer, mimeType is '**application/pdf**'
7	multipleSignedFileData	List<MultipleSignedFileData>	M/O	In case of signing multiple files, the multiple signed files will be responded in this object.
8	xmlSignature	String	M/O	XML Signature (XMLDSig or XML-XadDES)

2.8. prepareHashSigningForSignCloud

This method is known as "Sign Request", it means that client request to Remote Signing. Remote Signing will send back an Authorize Code via SMS or Email to authorize the client. This method also requires the hash value which needs to be signed.



2.8.1. Prototype

```
SignCloudResp prepareHashSigningForSignCloud(SignCloudReq signCloudReq);
```

2.8.2. Parameters

SignCloudReq Attributes				
No	Name	Type	Require	Description
1	relyingPartyBillCode	String	O	Relying Party's billcode
2	relyingParty	String	M	In case of many services are running on Remote Signing. RelyingParty is used to determine which client is calling to.
3	agreementUUID	String	M	Identify the client/customer/agreement ID
4	mimeType	String	M	The available mime type for hash signing: application/sha1-binary application/sha256-binary application/sha384-binary application/sha512-binary
5	hash	String	M	Hash to be signed

SignCloudReq Attributes				
No	Name	Type	Require	Description
6	hashAlgorithm	String	O	Hash algorithm. The default value is SHA256 The available mime type for hash signing: SHA-1; SHA-256; SHA-384; SHA-512
7	certificateRequired	Boolean	O	If certificateRequired is TRUE. The signing certificate is also responded
8	multipleSigningFileData	List<MultipleSigningFileData>	M/O	In case of multiple hashes need to be signed by one time authentication. This object contains a list of hashes will be signed.
9	notificationTemplate	String	M	Message contains Authorize Code will be sent to customer's Phone/Email. Authorize Code is generated on Remote Signing and embedded into template. E.g: Your authorize code: **{AuthorizeCode}**. Authorize Code is valid within 5 minutes.
10	notificationSubject	String	M/O	This parameter is used as Email subject for signing authorization. It should be used in case of the Remote Signing will use owned SMTP to send OTP Email to customer.
11	authorizeMethod	int	O	Which method will be used to authorize to client. 1: OTP SMS (default) 2: OTP Email 3: OTP Mobile 4: Passcode 5: UAF

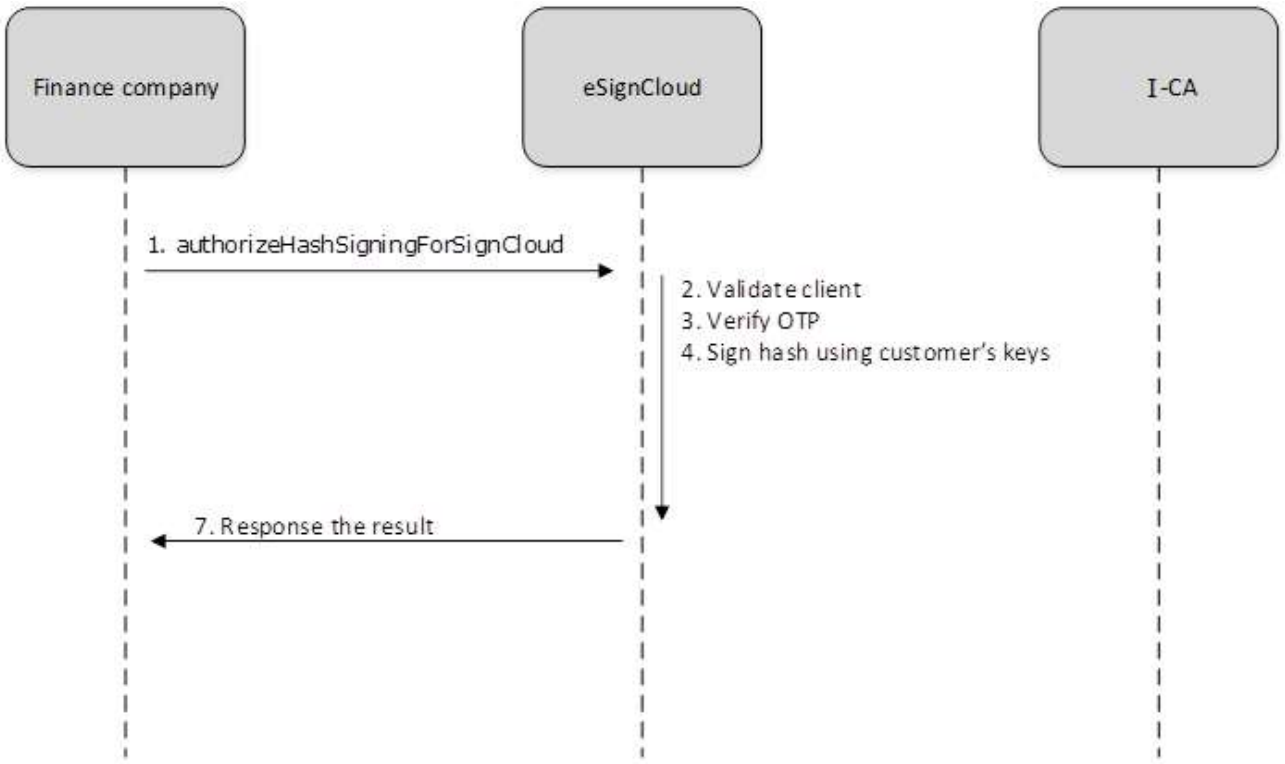
SignCloudReq Attributes				
No	Name	Type	Require	Description
				This attribute will be deprecated in the future and replaced by authMode
12	authorizeCode	String	M/O	Authorize Code provided by customer In this case of passcode (4) is used, authorize code is passcode
13	credentialData	CredentialData	M	Credential information used for client-server handshake

SignCloudResp Attributes				
No	Name	Type	Require	Description
1	responseCode	int	M	Code describes the result.
2	responseMessage	String	M	Message describes the result in detail.
3	billCode	String	M/O	Receipt for each transaction
4	notificationMessage	String	M/O	In case of Authorize Code isn't sent by Remote Signing, notification message which has already included Authorize Code will be sent back to Finance/Insurance company to send to their end-user.
5	notificationSubject	String	M/O	This parameter is used as Email subject for signing authorization. It should be used in case of RelyingParty used their owned SMTP server for notify customer
6	authorizeCredential	String	M/O	In case of Authorize Code isn't sent by Remote Signing, Email/Mobile number of client/customer also is responded back to Finance/Insurance company

SignCloudResp Attributes				
No	Name	Type	Require	Description
				and they use that information to send Authorize Code to their end-user.
7	remainingCounter	int	M	Counter decreases once invalid Authorization Code provided. AgreementUUID will be blocked if counter goes to zero and it's automatically unblocked within 5 minutes. The default value for automatic unblock is 5 minutes, it could be reconfigured in the system
8	signatureValue	String	M/O	The PKCS#1 Signature as Base64 String will be sent to RelyingParty
9	multipleSignedFileData	List<MultipleSignedFileData>	M/O	In case of signing multiple hashes, the multiple signatures will be responded in this object.

2.9. authorizeHashSigningForSignCloud

This method is known as "Sign Response", that means client responds the Authorize Code to Remote Signing. Remote Signing will verify the received Authorize Code and responses PKCS#1 signature whether it's correct. In case of the agreement don't own the certificate, the Remote Signing will enroll the corresponding ceritificate for business optimization and later will sign the hash.



2.9.1. Prototype

```
SignCloudResp authorizeHashSigningForSignCloud(SignCloudReq signCloudReq);
```

2.9.2. Parameters

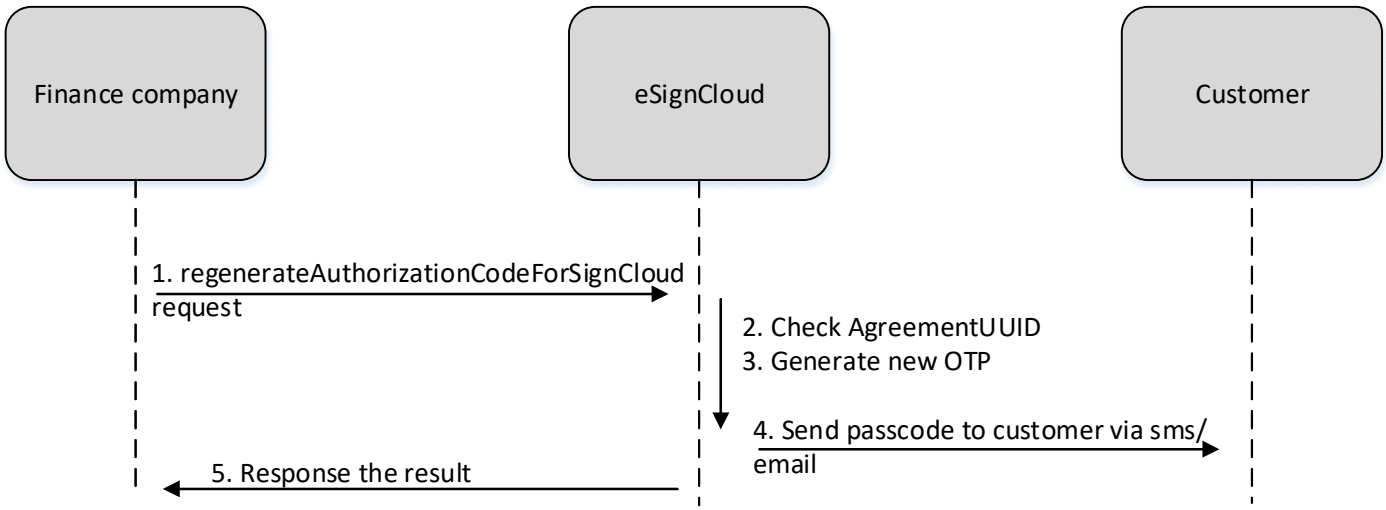
SignCloudReq Attributes				
No	Name	Type	Require	Description
1	relyingPartyBillCode	String	O	Relying Party's billcode
2	relyingParty	String	M	In case of many services are running on Remote Signing. RelyingParty is used to determine which client is calling to.
3	agreementUUID	String	M	Identify the client/customer/agreement ID
4	billCode	String	M	billCode obtained from response of **prepareHashSigningForSignCloud**
5	authorizeCode	String	M	Authorize Code provided by customer
6	messagingMode	int	O	Which messaging mode will be used **1**: ASYNCHRONOUS_CLIENTSERVER (default) **2**: ASYNCHRONOUS_SERVERSERVER **3**: SYNCHRONOUS If default value 0 is used, the API getSignatureValueForSignCloud will be used for getting the signature

SignCloudReq Attributes				
No	Name	Type	Require	Description
7	certificateRequired	Boolean	O	If you want signer certificate, set this attribute to True. The default is False
8	credentialData	CredentialData	M	Credential information used for client-server handshake

SignCloudResp Attributes				
No	Name	Type	Require	Description
1	responseCode	int	M	Code describes the result.
2	responseMessage	String	M	Message describes the result in detail.
3	billCode	String	M	Receipt for each transaction
4	remainingCounter	int	M	Counter decreases once invalid Authorization Code provided. AgreementUUID will be blocked if counter goes to zero and it's automatically unblocked within 5 minutes. The default value for automatic unblock is 5 minutes, it could be reconfigured in the system
5	signatureValue	String	M/O	The PKCS#1 Signature as Base64 String will be sent to RelyingParty
6	multipleSignedFileData	List<MultipleSignedFileData>	M/O	In case of signing multiple hashes, the multiple signatures will be responded in this object.

2.10. regenerateAuthorizationCodeForSignCloud

This function is used to regenerate the authorization code.



2.10.1. Prototype

```
SignCloudResp regenerateAuthorizationCodeForSignCloud(SignCloudReq signCloudReq);
```

2.10.2. Parameters

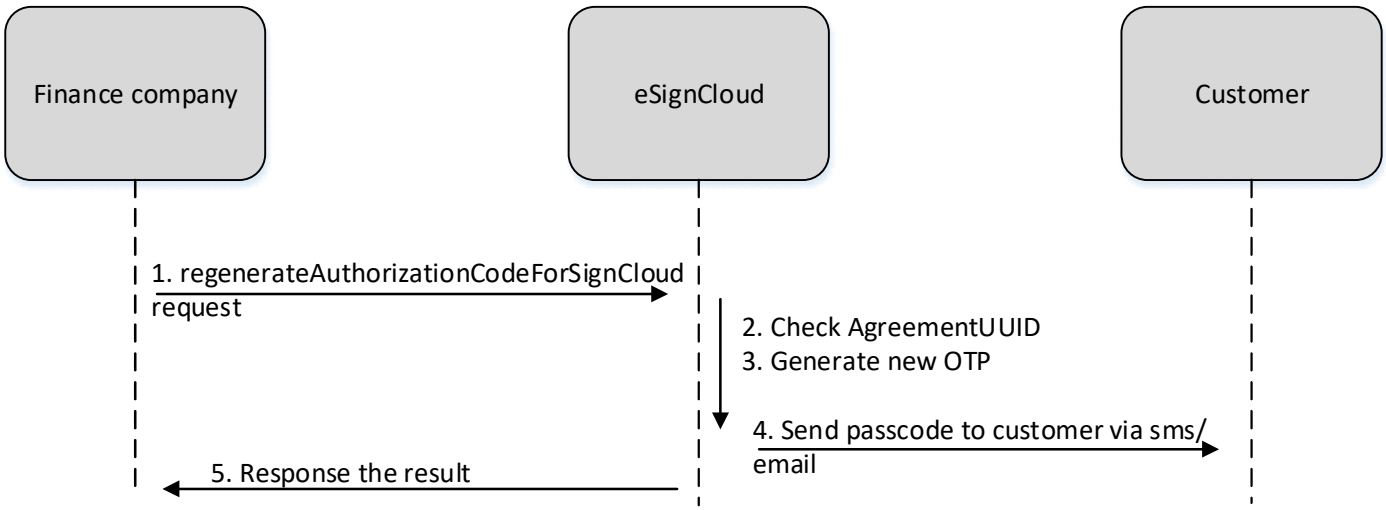
SignCloudReq Attributes				
No	Name	Type	Require	Description
1	relyingPartyBillCode	String	O	Relying Party's billcode
2	relyingParty	String	M	In case of many services are running on Remote Signing. RelyingParty is used to determine which client is calling to.
3	agreementUUID	String	M	Identify the client/customer/agreement number
4	notificationTemplate	String	M	Message contains Authorize Code will be sent to customer's Phone/Email. Authorize Code is generated on Remote Signing and embedded into template. E.g: Your authorize code: **{AuthorizeCode}**. Authorize Code is valid within 5 minutes.
5	notificationSubject	String	M/O	This parameter is used as Email subject for signing authorization. If OTP Email is used, this parameter is mandatory It should be used in case of the Remote Signing will use owned SMTP to send OTP Email to customer.
6	authorizeMethod	int	O	Which method will be used to authorize to client. 1: OTP SMS (default)

SignCloudReq Attributes				
No	Name	Type	Require	Description
				2: OTP Email 3: OTP Mobile 4: Passcode 5: UAF *This attribute will be deprecated in the future and replaced by authMode*

SignCloudResp Attributes				
No	Name	Type	Require	Description
1	responseCode	int	M	Code describes the result. Normally.
2	responseMessage	String	M	Message describes the result in detail.
3	billCode	String	M	Receipt for each transaction
4	notificationMessage	String	M/O	In case of Authorize Code isn't sent by Remote Signing, notification message which has already included Authorize Code will be sent back to Finance/Insurance company to send to their end-user.
5	notificationSubject	String	M/O	This parameter is used as Email subject for signing authorization. It should be used in case of RelyingParty used their owned SMTP server for notify customer
6	authorizeCredential	String	M/O	In case of Authorize Code isn't sent by Remote Signing, Email/Mobile number of client/customer also is responded back to Finance/Insurance company and they use that information to send Authorize Code to their end-user.

2.11. getSignedFileForSignCloud

This function is used to download signed documents which belong to a agreementUUID. Number of file can be determined by "filesLimit" attribute in request. If it isn't provided, 5 files will be the default.



2.11.1. Prototype

```
SignCloudResp getSignedFileForSignCloud(SignCloudReq signCloudReq);
```

2.11.2. Parameters

SignCloudReq Attributes				
No	Name	Type	Require	Description
1	relyingPartyBillCode	String	O	Relying Party's billcode
2	relyingParty	String	M	In case of many services are running on Remote Signing. RelyingParty is used to determine which client is calling to.
3	agreementUUID	String	M	Identify the client/customer/agreement number
4	billCode	String	O	Receipt for each transaction. If omitted, get the latest signed files of the agreement ID. The number of file is mentioned by "filesLimit"
5	filesLimit	int	O	Number of file will be responded. The default is 5 and maxium file allowed to download is 10
6	p2pEnabled	boolean	O	Must provide TRUE Once this parameter is TRUE, RP should download directly from Remote Signing service. Vice versa, it should be downloaded by DMS
7	credentialData	CredentialData	M	Credential information used for client-server handshake

SignCloudResp Attributes				
No	Name	Type	Require	Description
1	responseCode	int	M	Code describes the result. Normally.
2	responseMessage	String	M	Message describes the result in detail.
3	billCode	String	M	Receipt for each transaction
4	signedFileData	Byte[]	M/O	File binary that be signed (in case of only one signed file available)
5	multipleSignedFileData	List<MultipleSignedFileData>	M/O	In case of signing multiple files, the multiple signed files will be responded in this object. (in case of multiple signed files available)

2.12. requestAuthentication

Provide customer information such as:

- Full name
- Personal ID
- Address
- Mobile number (used for received SMS OTP)
- Both sides of scanned of identity document

After calling this function, customer will receive SMS OTP sent by Service Provider

2.12.1. Prototype

```
SignCloudResp requestAuthentication(SignCloudReq signCloudReq);
```

2.12.2. Parameter

SignCloudReq Attributes				
No	Name	Type	Require	Description
1	relyingPartyBillCode	String	O	Relying Party's billcode
2	relyingParty	String	M	In case of many services are running on Remote Signing. RelyingParty is used to determine which client is calling to.
3	agreementUUID	String	M	Identify the client/customer/agreement number
4	ownerUUID	String	O	System will find OWNER_ID based on ownerUUID and the newly registered

SignCloudReq Attributes				
No	Name	Type	Require	Description
				certificate will be belonged to this owner. If this value is NULL, new owner will be created.
5	ownerUsername	String	O	ownerUsername and ownerPassword are both required not NULL to take effect. Newly created owner will be assigned this username/password. Error will be returned if the ownerUsername existed.
6	ownerPassword	String	O	
7	mobileNo	String	O	Mobile Number of client/customer. Known as contact address and used for SMS Authorize Code authentication. In case of the Relying Party keep the customer information confidentially, it should be absence
8	email	String	O	Email of client/customer and used for Email Authorize Code authentication In case of the Relying Party keep the customer information confidentially, it should be absence
9	certificateProfile	String	O	The profile of certificate will be used to enroll. Once this parameter is absence, the default certificate profile is used based on RelyingParty properties
10	promotion	int	O	The extra free day will be added for end-user certificate instead of only the day configured in certificate profile.
11	signingProfile	String	O	The profile decides the number of signing times the end-user can use. This value will be decreased once user performs signing. User cannot sign if the signing counter is 0. If signing counter is -1, user's siging times is unlimited.
12	agreementDetails	AgreementDetails	M	Customer information. This one will be used as certificate information.

SignCloudReq Attributes				
No	Name	Type	Require	Description
13	sharedMode	int	O	Which sharing mode will be used in the Remote Signing service. 1: PRIVATE_MODE 2: RP_SHARED_MODE 3: AGREEMENT_SHARED_MODE Default value is PRIVATE_MODE. It's meant once Relying Party don't use this parameter, the PRIVATE_MODE is used instead
14	postbackEnabled	Boolean	O	If postbackEnabled set **TRUE**, the Remote Signing service will response **REQUEST ACCEPTED** immediately. Once the certificate enrollment process is completely finalized, the Remote Signing will send the details based on the postbackUrl that already configured in RelyingParty properties Default value is **FALSE**, Postback mechanism shouldn't be used
15	csrRequired	Boolean	O	If csrRequired set **TRUE**, Remote Signing service will generate keypair for customer and build the CSR for the response.
16	credentialData	CredentialData	M	Credential information used for client-server handshake Cached feature is turn **ON/OFF**. Once the cached machine is **ON**, the Remote Signing just checked the credentialData in the handshake phase. Once credentialData is valid, our session will be valid in duration of 1 hour (this parameter could be reconfigured in Remote Signing service). The Relying Party need to handshake again before the session is expired

In case of AgreementDetails, (*) is mandatory and (/*) is at least ONE in mandatory

AgreementDetails Attributes				
No	Name	Type	Require	Description
1	**personName (*)**	String	M/O	Name of customer
2	organization	String	M/O	Company name
3	organizationUnit	String	M/O	Department nam
4	title	String	M/O	Title of employee
5	email	String	M/O	Personal email
6	telephoneNumber	String	M/O	Personal phone number
7	**location (*)**	String	M	
8	**stateOrProvince (*)**	String	M	City or Province
9	country	String	M	Country (Just two letter) E.g: VN
10	**personalID (/*)**	String	M/O	Personal ID
	passportID (/*)	String	M/O	Personal Passport ID
11	taxID (/*)	String	M/O	Tax ID
	budgetID (/*)	String	M/O	Budget ID
12	applicationForm	Byte[]	M/O	Remote Signing Service application form
13	requestForm	Byte[]	M/O	Remote Signing Service request form for recovery, revocation …
14	authorizeLetter	Byte[]	M/O	Authorize Letter by Government/Corporate
15	photoIDCard	Byte[]	M/O	Scanned photo of ID Card that applied for personal certificate (In case only 1 file available)
16	photoFrontSideIDCard	Byte[]	M/O	Scanned front side of photo of ID Card that applied for personal certificate
17	photoBackSideIDCard	Byte[]	M/O	Scanned back side side of photo of ID Card that applied for personal certificate
18	photoActivityDeclaration	Byte[]	M/O	Scanned photo of 'Initial Activity/Alteration Declation' that applied for corporate certificate

SignCloudResp Attributes				
No	Name	Type	Require	Description
1	responseCode	int	M	Code describes the result. Normally. Remote Signing will response **SUCCESSFULLY**
2	responseMessage	String	M	Message describes the result in detail.
3	billCode	String	M	Receipt for each transaction
4	certificateDN	String	M/O	Certificate Distinguished Name (DN)

SignCloudResp Attributes				
No	Name	Type	Require	Description
				E.g: CN=Nguyen Van A, O=B Company,C=VN
5	certificateSerialNumber	String	M/O	Certificate serial number E.g: 5405ABCDEF
6	certificateThumbprint	String	M/O	Certificate thumprint (certificate hash)
7	validFrom	Date	M/O	The date of certificate begin to be valid
8	validTo	Date	M/O	The date of certificate begin to be expired
9	issuerDN	String	M/O	Issuer Certifcate Distinguished Name (DN) ***E.g: CN=ICA Certification Authority, O=I-CA,C=VN***
10	certificate	String	M/O	The data of certificate in PEM format
11	certificateStateID	int	M/O	Certificate state ID • INITIALIZED = 2 • GENERATED = 3 • OPERATED = 4 • REVOKED = 5 • EXPIRED = 6 • DECLINED = 7 • RENEWED = 8 • RENEWED_EXPIRED = 9 • REVISED = 10 • RENEWED_KEEP_SN = 11 • BLOCKED = 12 REVISED_KEEP_SN = 13
12	signingCounter	int	M/O	How many times the certificate is used for signing. The default value is 1. For document signing this value should be greater than 1

CredentialData Attributes				
No	Name	Type	Require	Description
1	username	String	M	Username of relyingParty provided by Remote Signing
2	password	String	M	Password of relyingParty provided by Remote Signing
3	signature	String	M	Signature of relyingParty provided by Remote Signing

CredentialData Attributes				
No	Name	Type	Require	Description
4	pkcs1Signature	String	M	Value is signed from client private key. Key is generated and provided by Remote Signing Value=username + password + signature + timestamp
5	timestamp	String	O	Current timestamp, format in yyyyddmmHHMMss or epoch time If timestamp is absence, the pkcs1Signature is fixed for all transactions

2.13. respondAuthentication

OTP verification will be performed in this function. Server will record the OTP validity, no any certificate issued in this function.

2.13.1. Prototype

```
SignCloudResp respondAuthentication(SignCloudReq signCloudReq);
```

2.13.2. Parameter

SignCloudReq Attributes				
No	Name	Type	Require	Description
1	relyingPartyBillCode	String	O	Relying Party's billcode
2	relyingParty	String	M	In case of many services are running on Remote Signing. RelyingParty is used to determine which client is calling to.
3	agreementUUID	String	M	Identify the client/customer/agreement ID
4	billCode	String	M	billCode obtained from response of **requestAuthentication**
5	authorizeCode	String	M	Authorize Code provided by customer
6	credentialData	CredentialData	M	Credential information used for client-server handshake

SignCloudResp Attributes				
No	Name	Type	Require	Description
1	responseCode	int	M	Code describes the result.
2	responseMessage	String	M	Message describes the result in detail. Normally, this responses **SUCCESSFULLY**

SignCloudResp Attributes				
No	Name	Type	Require	Description
3	billCode	String	M	Receipt for each transaction
4	remainingCounter	int	M	Counter decreases once invalid Authorization Code provided. AgreementUUID will be blocked if counter goes to zero and it's automatically unblocked within 5 minutes. The default value for automatic unblock is 5 minutes, it could be reconfigured in the system

2.14. approveAuthentication

Once this API is called, server will check the previous state of OTP authentication. If the OTP is valid, customer certificate will be generated and the document will be signed.

Document information should be provided in this API. Server also generates the PDF file based on that information.

The signed document could be returned by many ways: SFTP, ICA or synchronously in the response.

2.14.1. Prototype

```
SignCloudResp approveAuthentication(SignCloudReq signCloudReq);
```

2.14.2. Parameter

SignCloudReq Attributes				
No	Name	Type	Require	Description
1	relyingPartyBillCode	String	O	Relying Party's billcode
2	relyingParty	String	M	In case of many services are running on Remote Signing. RelyingParty is used to determine which client is calling to.
3	agreementUUID	String	M	Identify the client/customer/agreement ID
4	billCode	String	M	billCode obtained from response of **requestAuthentication**
6	messagingMode	int	O	Which messaging mode will be used **1**: ASYNCHRONOUS_CLIENTSERVER (default) **2**: ASYNCHRONOUS_SERVERSERVER **3**: SYNCHRONOUS

SignCloudReq Attributes				
No	Name	Type	Require	Description
				If default value 0 is used, the API downloadSignedFileForSignCloud will be used for getting the signed file
7	credentialData	CredentialData	M	Credential information used for client-server handshake
8	templateId	String	M	Template to generate PDF
9	signCloudMetaData	signCloudMetaData	O	SignCloudMetaData contains additional signer properties which are used for PDF signature display. - Coordinate - Background image - Signer location - Signer reason - Text color In case of XML sigining, it will indicate which type should be applied XMLDSig, XML-XaDES, signature zone Once this parameter is not used, Remote Signing should use the default value that already configured in the system In case of counter signing process, SignCloudMetaData included 2 MetaDatas for customer and Relying Party signer properties
10	documentAttributes	DocumentAttributes	M	Document content to be filled into template

SignCloudResp Attributes				
No	Name	Type	Require	Description
1	responseCode	int	M	Code describes the result.
2	responseMessage	String	M	Message describes the result in detail. Normally, this responses **SUCCESSFULLY**
3	billCode	String	M	Receipt for each transaction
4	signedFileData	Byte[]	M/O	File binary that be signed

SignCloudResp Attributes				
No	Name	Type	Require	Description
5	mimeType	String	M/O	In case of PDF file that be downloaded, mimeType is '**application/pdf**'
6	signedFileUUID	String	M/O	Signed file UUID that identified by our Remote Signing service
7	multipleSignedFileData	List<MultipleSignedFileData>	M/O	In case of signing multiple files, the multiple signed files will be responded in this object.
8	xmlSignature	String	M/O	XML Signature (XMLDSig or XML-XadDES)

DocumentAttributes Attributes				
No	Name	Type	Require	Description
1	templateName	String	M	Template name
2	attributes	HashMap<String,String>	M	Document attributes are used to generate PDF

2.15. uploadFileForSignCloud

This method will upload file based on Remote Signing service

2.15.1. Prototype

```
SignCloudResp uploadFileForSignCloud(SignCloudReq signCloudReq);
```

2.15.2. Parameters

SignCloudReq Attributes				
No	Name	Type	Require	Description
1	relyingPartyBillCode	String	O	Relying Party's billcode
2	relyingParty	String	M	In case of many services are running on Remote Signing. RelyingParty is used to determine which client is calling to.
3	agreementUUID	String	M	Identify the client/customer/agreement ID
4	agreementDetails	AgreementDetails	M	2 sides of scanned ID document of customer will be presented in this object.

SignCloudReq Attributes				
No	Name	Type	Require	Description
5	credentialData	CredentialData	M	Credential information used for client-server handshake

SignCloudResp Attributes				
No	Name	Type	Require	Description
1	responseCode	int	M	Code describes the result.
2	responseMessage	String	M	Message describes the result in detail.
3	billCode	String	M	Receipt for each transaction

AgreementDetails Attributes				
No	Name	Type	Require	Description
12	applicationForm	Byte[]	M/O	Remote Signing Service application form
13	requestForm	Byte[]	M/O	Remote Signing Service request form for recovery, revocation …
14	authorizeLetter	Byte[]	M/O	Authorize Letter by Government/Corporate
15	**photoIDCard**	**Byte[]**	**M/O**	**Scanned photo of ID Card that applied for personal certificate (In case only 1 file available)**
16	**photoFrontSideIDCard**	**Byte[]**	**M/O**	**Scanned front side of photo of ID Card that applied for personal certificate**
17	**photoBackSideIDCard**	**Byte[]**	**M/O**	**Scanned back side side of photo of ID Card that applied for personal certificate**
18	photoActivityDeclaration	Byte[]	M/O	Scanned photo of 'Initial Activity/Alteration Declation' that applied for corporate certificate

3.RESPONSE CODE AND MESSAGE

0	SUCCESSFULLY
1000	INVALID IP ADDRESS
1001	INVALID CREDENTIAL DATA
1002	INVALID PARAMS
1003	UNEXPECTED EXCEPTION
1004	INVALID AUTHORIZATION CODE
1005	AUTHORIZATION BLOCKED
1006	AUTHORIZATION CODE TIMEOUT
1007	REQUEST ACCEPTED
1008	AGREEEMENT NOT FOUND
1009	CERTIFICATE IS NOT ENROLLED
1010	AGREEMENT EXISTED
1011	UNSUPPORTED OPERATION
1012	ACCESS DENIED
1013	AGREEMENT NOT READY
1014	CERTIFICATE IS EXPIRED
1015	BILLCODE TIMEOUT DUE TO UNEXPECTED EXCEPTION WHILE SIGNING
1016	FILE IS BEING PROCESSED
1017	AGREEMENT REVOKED
1018	SUCCESSFULLY. YOUR PASSCODE NEED TO BE CHANGED
1019	FUNCTION ACCESS DENIED
1020	FAILED TO CHECK REVOCATION
1021	CERTIFICATE REVOKED
1022	CERTIFICATE UNKNOWN
1023	SHARED AGREEMENT NOT FOUND
1024	SHARED AGREEMENT INVALID SHARED MODE
1025	FILE NOT FOUND
1026	MULTIPLE CLIENT MACHINE YSING THE SAME AGREEMENT UUID
1027	SIGNING TRANSACTION CAN NOT BE COMMITED SINCE SIGNING ERROR
1028	CREDENTIAL IS EXPIRED
1029	EMAIL IS ALREADY REGISTERED
1030	DOCUMENT IS NOT SIGNED YET
1031	CERTIFICATE IS NOT ASSIGNED TO OWNER
1032	USERNAME IS ALREADY EXISTED
1033	SIGNING COUNTER IS EXCEEDED
1034	ENTERPRISE CODE IS EXISTED
1035	PERSONAL CODE IS ALREADY EXISTED

1036	CERTIFICATE ISSUANCE ERROR
1037	MOBILE APP SESSION IS EXPIRED. PLEASE AUTHORIZE AGAIN
1038	SAD IS INVALID
1039	NUMBER OF ALLOWED OTP GENERATION EXCEEDED
1040	INVALID AUTH METHOD
1041	THE DEFAULT PASSCODE MUST BE CHANGED
1042	AGREEMENT HAS NOT PERFORMED SIGNING YET
1043	IDENTITY DOCUMENT REQUIRED
1044	AUTHENTICATION FAILED
1045	FAILED TO SEND OTP
1046	SIGNER INFO REQUIRED FOR CHECKING
1047	INVALID SIGNER INFO PROVIDED
1048	NO CERTIFICATE FOUND
1049	NO KEY FOUND
1050	SIGNATURE POSITION NOT FOUND

--END--

APPENDIX A: CREDENTIAL DATA DETAILS

Create PKCS#1 signature in Java

```
import java.io.FileInputStream;
import java.io.InputStream;
import java.security.KeyStore;
import java.security.PrivateKey;
import java.security.Signature;
import java.util.Enumeration;
--------------------------------------------------------------------
KeyStore keystore = KeyStore.getInstance("PKCS12");
InputStream is = new FileInputStream("cloudfca.p12");
keystore.load(is, "12345678".toCharArray());

Enumeration<String> e = keystore.aliases();
String aliasName = "";
PrivateKey key = null;
while (e.hasMoreElements()) {
      aliasName = e.nextElement();
      key = (PrivateKey) keystore.getKey(aliasName, "12345678".toCharArray());
      if(key != null)
            break;
}
Signature sig = Signature.getInstance("SHA1withRSA");
sig.initSign(key);
sig.update("To be signed".getBytes());
String pkcs1Signature = DatatypeConverter.printBase64Binary(sig.sign());
```

Create PKCS#1 signature in Python

```
# easy_install.exe http://www.voidspace.org.uk/python/pycrypto-2.6.1/pycrypto-
2.6.1.win-amd64-py2.7.exe
# python get-pip.py
# python -m pip install --upgrade pip
# pip install pyopenssl
from Crypto.Signature import PKCS1_v1_5
from Crypto.Hash import SHA
from Crypto.PublicKey import RSA
from OpenSSL import crypto
from _helpers import _parse_pem_key
from _helpers import _to_bytes
from _helpers import _b64encode

p12 = crypto.load_pkcs12(open("cloudfca.p12", 'rb').read(), "12345678")
p12.get_certificate()     # (signed) certificate object
p12.get_privatekey()      # private key.
p12.get_ca_certificates() # ca chain.

pKeyPem = crypto.dump_privatekey(crypto.FILETYPE_PEM, p12.get_privatekey())

parsed_pem_key = _parse_pem_key(_to_bytes(pKeyPem))

message = 'To be signed'
key = RSA.importKey(parsed_pem_key)
h = SHA.new(message)
signer = PKCS1_v1_5.new(key)
signature = signer.sign(h)
print("Signature: "+_b64encode(signature))
```

Helper class

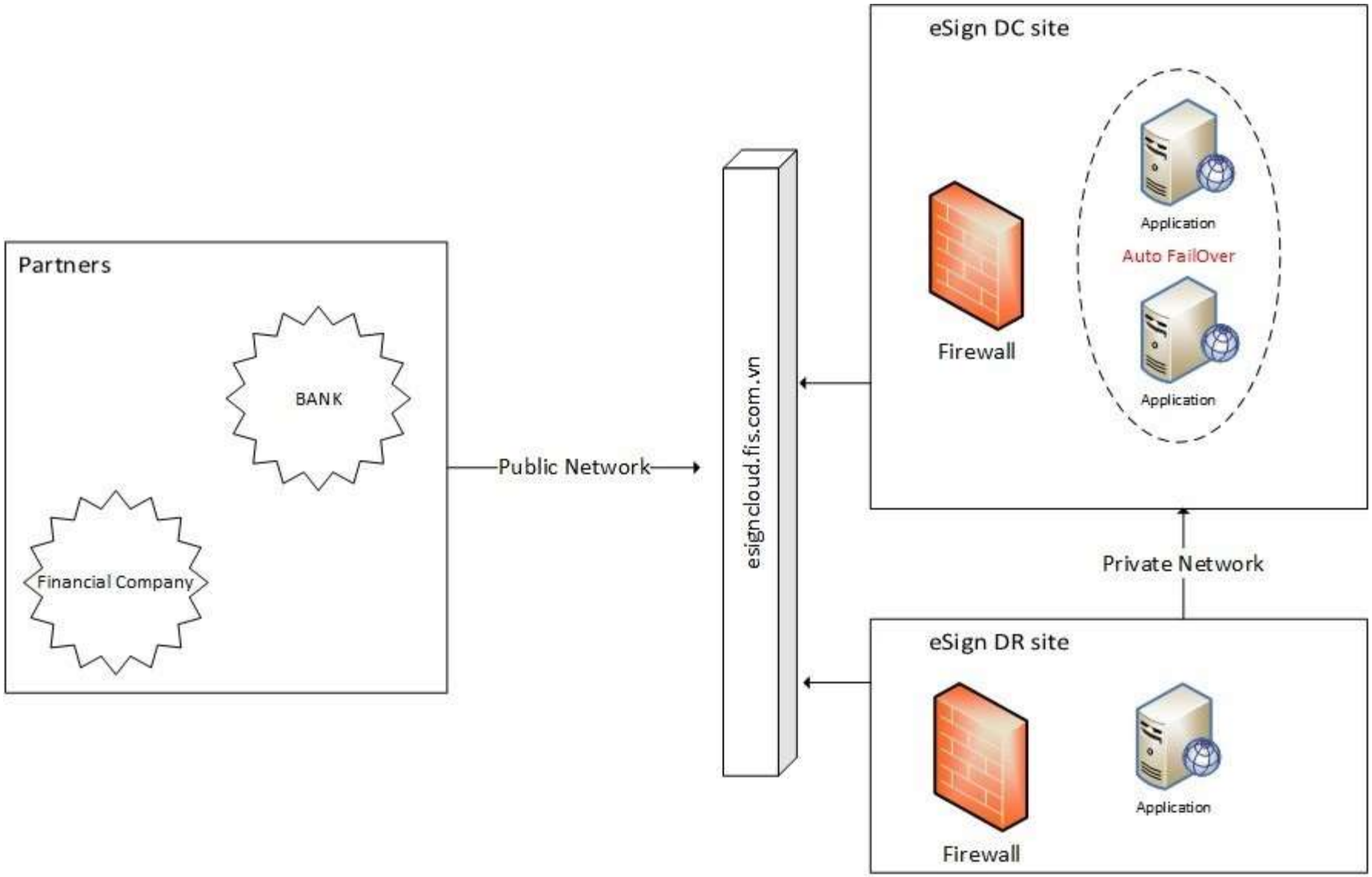
```
import base64
import json
import six

def _parse_pem_key(raw_key_input):
    """Identify and extract PEM keys.
    Determines whether the given key is in the format of PEM key, and extracts
    the relevant part of the key if it is.
    Args:
        raw_key_input: The contents of a private key file (either PEM or
                       PKCS12).
    Returns:
        string, The actual key if the contents are from a PEM file, or
        else None.
    """
    offset = raw_key_input.find(b'-----BEGIN ')
    if offset != -1:
        return raw_key_input[offset:]

def _to_bytes(value, encoding='ascii'):
    result = (value.encode(encoding)
              if isinstance(value, six.text_type) else value)
    if isinstance(result, six.binary_type):
        return result
    else:
        raise ValueError('%r could not be converted to bytes' % (value,))

def _b64encode(raw_bytes):
      raw_bytes = _to_bytes(raw_bytes, encoding='utf-8')
      return base64.b64encode(raw_bytes)
```

APPENDIX B: INTEGRATED ARCHITECTURE



APPENDIX C: SIGNCLOUD METADATA DETAILS

SignCloudMetaData has two attributes

- singletonSigning: Define the signature properties for customer
- counterSigning: Define the signature properties for organization

Both singletonSigning and counterSigning are Key-Value objects.

KEY	EXAMPLE VALUE	DESCRIPTION
PAGENO	First;Last;1;2;..	The page, signature will be put on. Possible value: 1; 2; 3…; First; Last
POSITIONIDENTIFIER	CHỮ KÝ KHÁCH HÀNG	Signature will be placed based on the detected string. Ex: "Customer signature" Server will find the coordinates of "Customer signature" and put the signature based on that coordinates.
RECTANGLEOFFSET	-10,120	This will be used with POSITIONIDENTIFIER. Let say POSITIONIDENTIFIER is "Customer signature" and the found coordinate is of "C" character. RECTANGLEOFFSET is the offset from "C" will help us correctly put the signature on.
RECTANGLESIZE	170,70	The size of rectangle signature
VISIBLESIGNATURE	True;False	True – Signature is visible False – Signature is invisible
VISUALSTATUS	True;False	True/False – Visiable/Hide the green stick validation
IMAGEANDTEXT	True;False	Showing signer info and image (logo) at the same time
TEXTDIRECTION		Used with IMAGEANDTEXT. Possible value • LEFTTORIGHT • RIGHTTOLEFT • TOPTOBOTTOM • BOTTOMTOTOP • OVERLAP
SHOWSIGNERINFO	True;False	Showing signer info – CN of certificate

SIGNERINFOPREFIX	Sign by:	The prefix of signer info. Ex: **Sign by:** <CN>
SHOWDATETIME	True;False	Showing signing datetime
DATETIMEPREFIX	Sign on:	The prefix of signing datetime. Ex: **Sign on:** <timestamp>
DATETIMEFORMAT	dd-MM-yyyy HH:mm:ss	Defining datetime format. Ex: dd-MM-yyyy HH:mm:ss
SHOWREASON	True;False	Showing signing reason
SIGNREASONPREFIX	Sign for:	The prefix of reason. Ex: **Sign for:** Approval
SHOWLOCATION	True;False	Showing signing location
LOCATION	HCM	Signing location. Ex: HCM
LOCATIONPREFIX	Sign at:	The prefix of location. Ex: **Sign location:** HCM
TEXTCOLOR	back;red	Text color. Possible value: yellow; blue; cyan; dark_gray; gray; green; light_gray; magenta; orange; pink; red; white
SHOWTELEPHONE	True;False	Showing telephone (telephone field of certificate)
TELEPHONEPREFIX	Điện thoại:	The prefix of telephone. Ex: **Điện thoại:** 090888888
SHOWENTERPRISEID	True;False	Showing taxID of organization (find MST: or MNS: in certificate)
ENTERPRISEIDPREFIX	MST	The prefix of taxID of organization. Ex: **MST:** 111111
SHOWPERSONALID	True;False	Showing personal ID (find CMND, CCCD or HC in certificate)
PERSONALIDPREFIX	CMND/CCCD	The prefix of personal ID. Ex: CMND: 222222
SHOWSERIALNUMBER	True;False	Showing certificate serial number
SERIALNUMBERPREFIX	SN:	The prefix of certificate serial number
SHOWORGANIZATION	True;False	Showing organization (O field of certificate)
ORGANIZATIONPREFIX	Tổ chức:	The prefix of organization
SHOWTITLE	True;False	Showing title (T field of certificate)
TITLEPREFIX	Chức vụ:	The prefix of title
ONLYCOUNTERSIGNENABLED	True;False	If True, only organization signature is applied.
COUNTERAGREEMENTUUID		Organization agreementUUID

COUNTERAGREEMENTPASSCODE		Organization passcode
COUNTERSIGNENABLED	True;False	If True, Organization and Customer signature are both applied.
SHOWISSUERINFO	True;False	Showing Issuer name of certificate
ISSUERINFOPREFIX		The prefix of issuer name
BACKGROUNDIMAGE	Base64 of image	Background image of signature field. Image will be resized to fix the signature rectangle.
PAGESFORINITIALSIGNATURE	all;1,2,3..	Initial signature
SHADOWSIGNATUREPROPERTIES	all	Multiple signatures fields for one time authentication
PDFPASSWORD	12345678	PDF protected password
LINESPACING	1.5	Space between lines
FONTSIZE	12	Font size
FONTSTYLE	• ARIAL • TAHOMA • TIMES • VERDANA • SCRIPTURE • HELVETICA	Font style

APPENDIX D: SERVICE SECURITY

- **Encryption:**
  - Using HTTPS to secure the transferring data on the network

- **Authentication:**
  - Using username/password for entity authentication
  - Using client private key to sign an client identity evidence. Server keeps entity's public key to verify the client signature. The signature is valid, the identity of entity is proven. *(Signature algorithm is SHA1withRSA)*
  - IP filter: Client's IPs should be whitelisted unless all requests will be rejected (Configured on server side)
  - Function access restriction: Client only calls the APIs that are granted by vendor (Configured on server side)